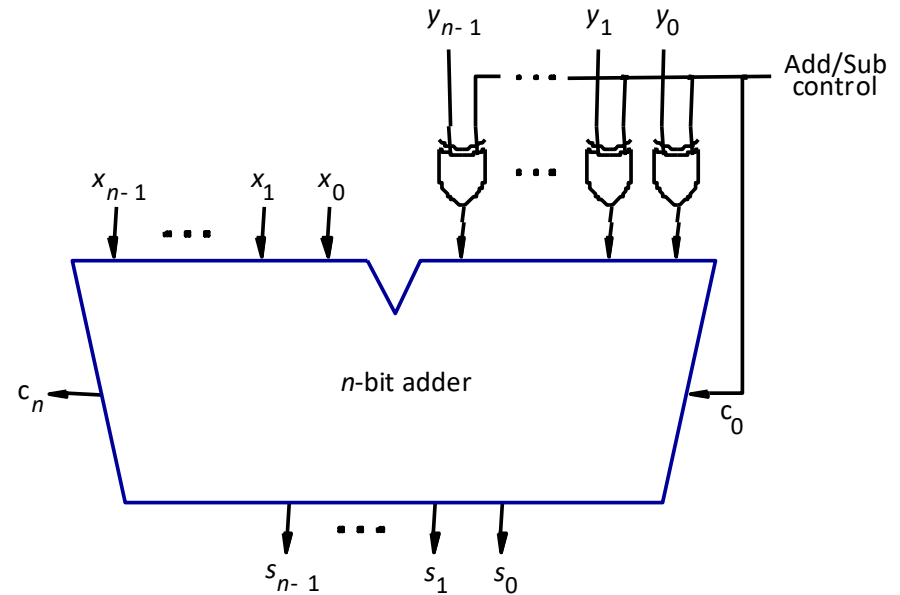


Lecture 7

Processing Unit

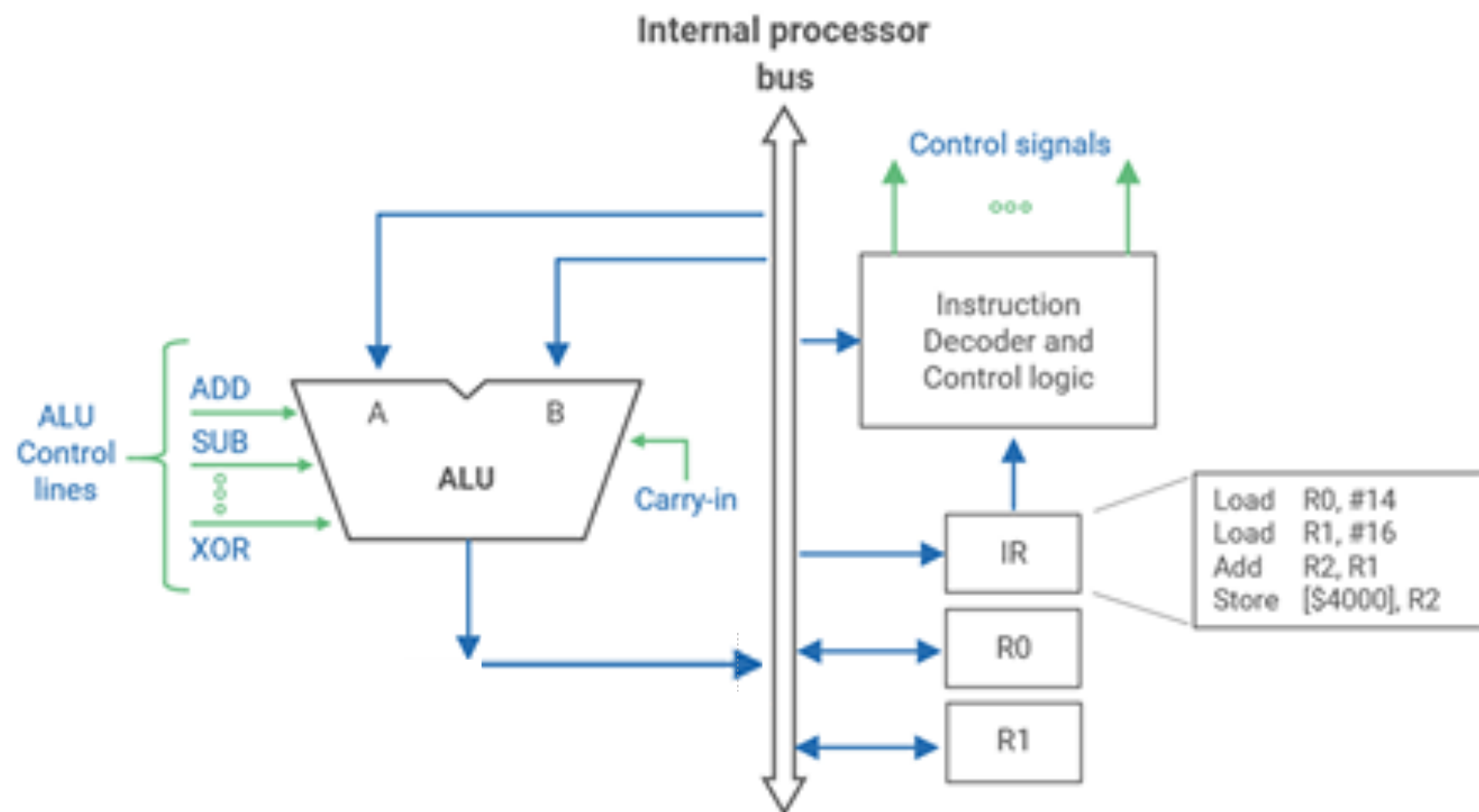
Previously ..

- Add
 - FA
 - Carry Ripple Adder
 - Carry Lookahead Adder
- Multiply
 - Booth Algorithm
 - Fast Multiplier with Bit-pair and Carry Save
- Divide
 - Restoring and Non-restoring
- Keep in mind that we learned integer arithmetic operations



- Add/sub control = 0, addition.
- Add/sub control = 1, subtraction.

Central Processing Unit



Basic of Executing an Instruction (1)

- A processor fetches one instruction at a time and performs a required operation.
- Instructions are fetched from successive memory locations, until a branch instruction is encountered.
- The PC is used to keep track of instruction sequence.
- Instruction Register (IR)

Basic of Executing an Instruction (2)

- Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (**fetch stage**).

$$IR \leftarrow ([PC])$$

- Assuming that the memory is byte addressable, increment the contents of the PC by 4 (**fetch stage**).

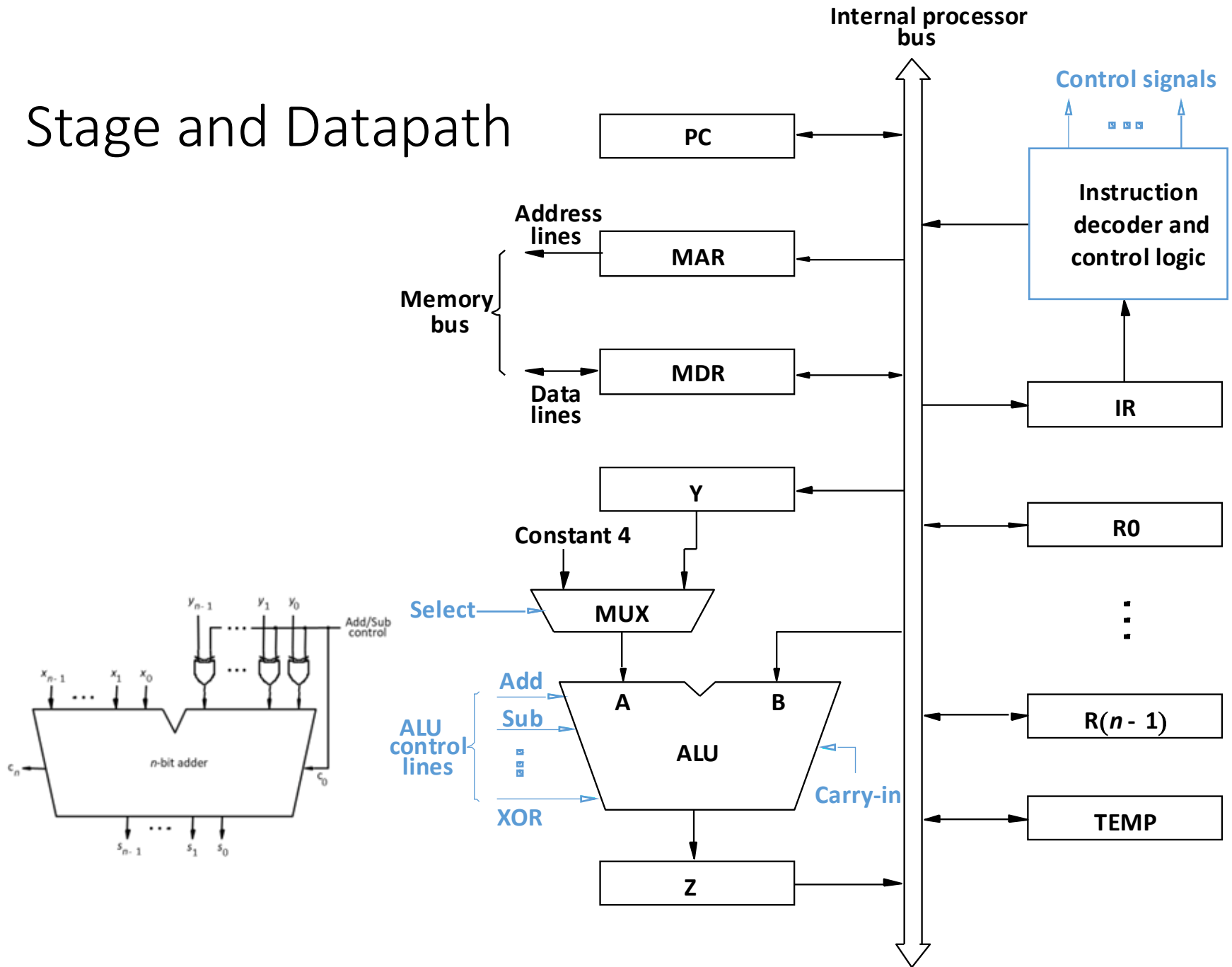
$$PC \leftarrow [PC] + 4$$

- Carry out the actions specified by the instruction in the IR (**execution stage**).

(1) Stage and Datapath

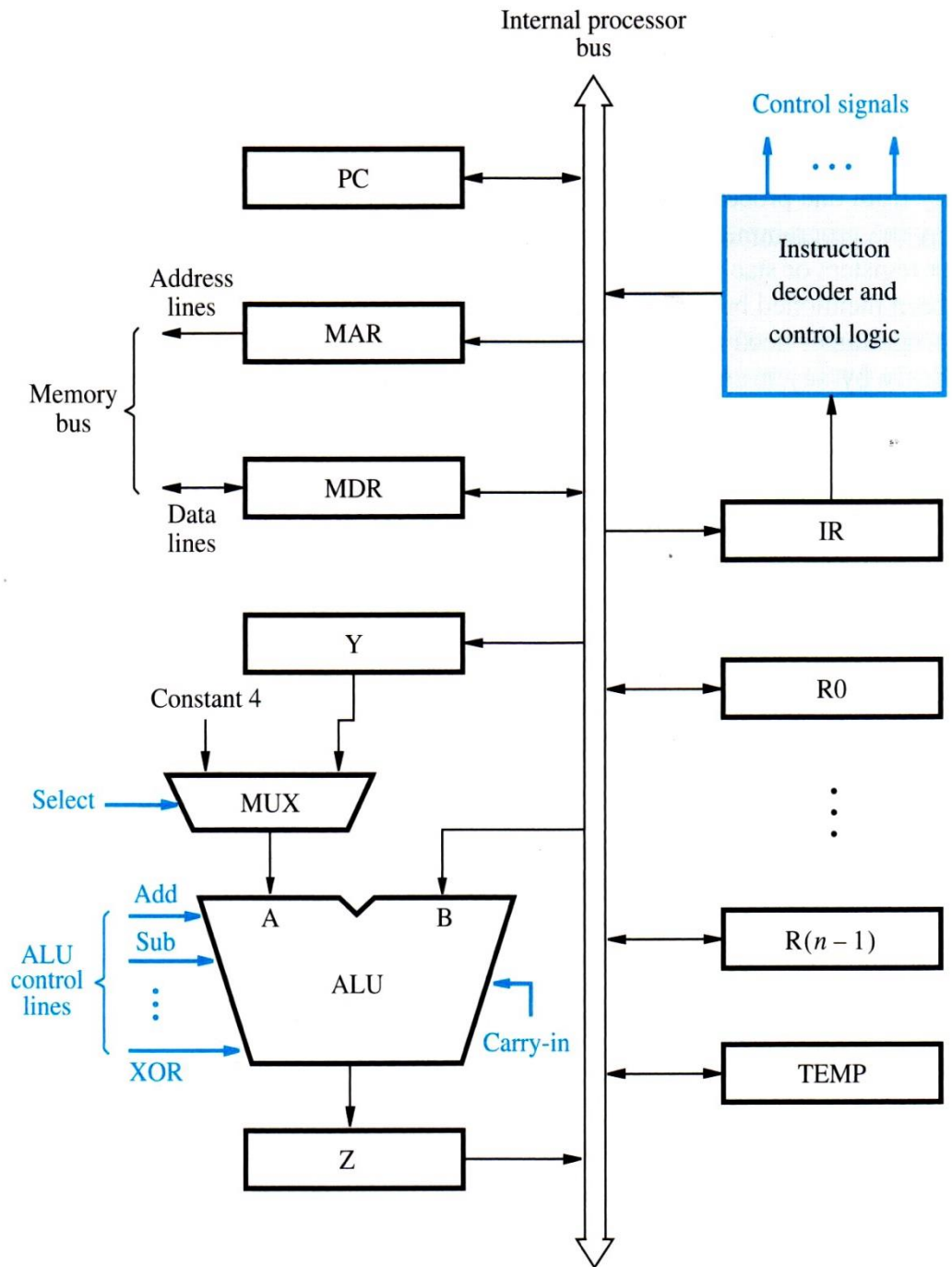
(2) Control Unit

Stage and Datapath



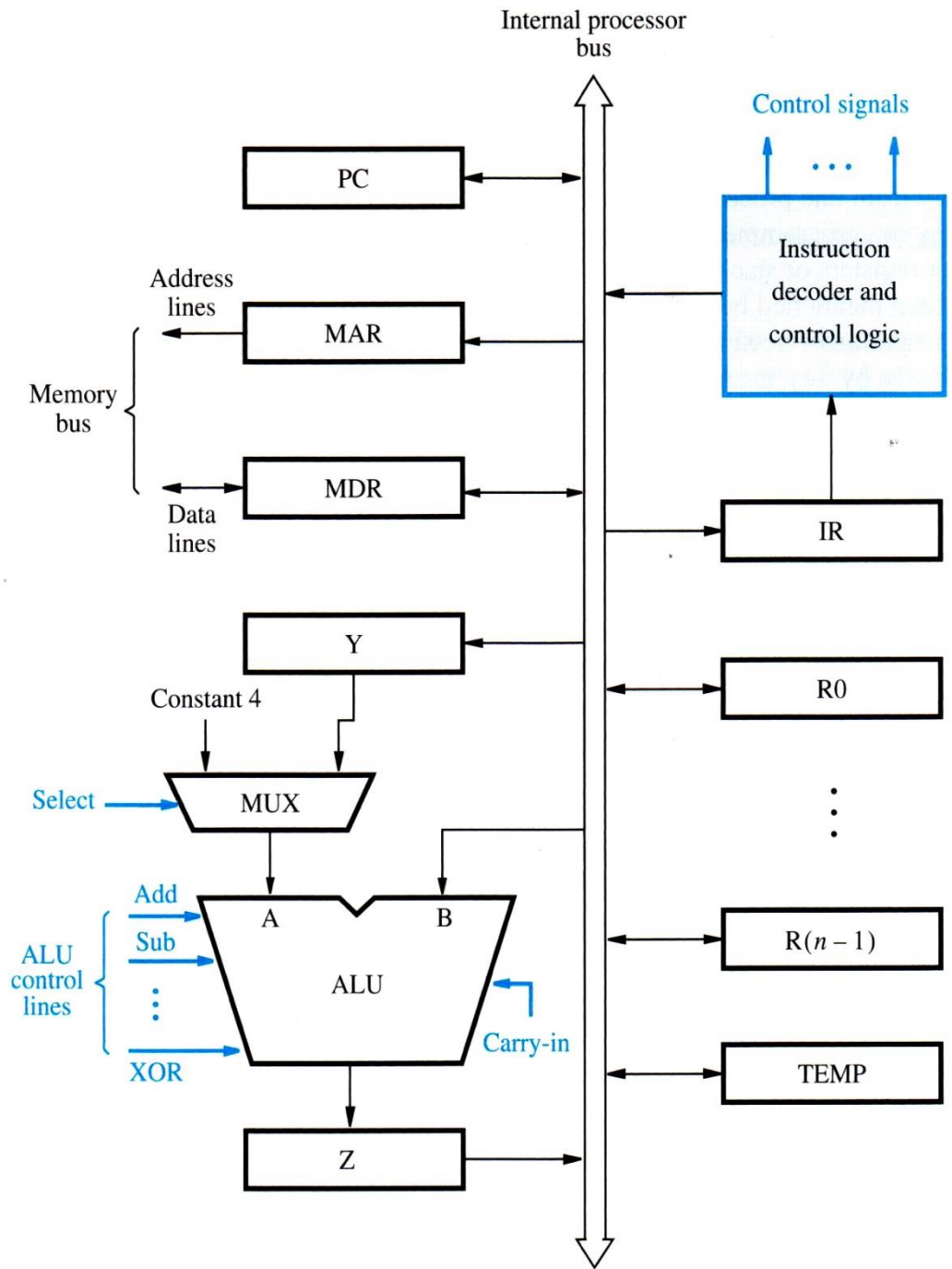
Fundamental Concepts (1)

- Every component is connected via a **bus**.
- Y, Z, and TEMP are registers that are **transparent** to the programmer. They are never referenced explicitly.
- MUX chooses to increment the program counter. It chooses between 4 or the content of Y!



Fundamental Concepts (2)

- The instruction decoder and control logic is responsible for **decoding** and implementing **the action** specified in the IR. The unit selects registers involved and directs **the transfer of data**.

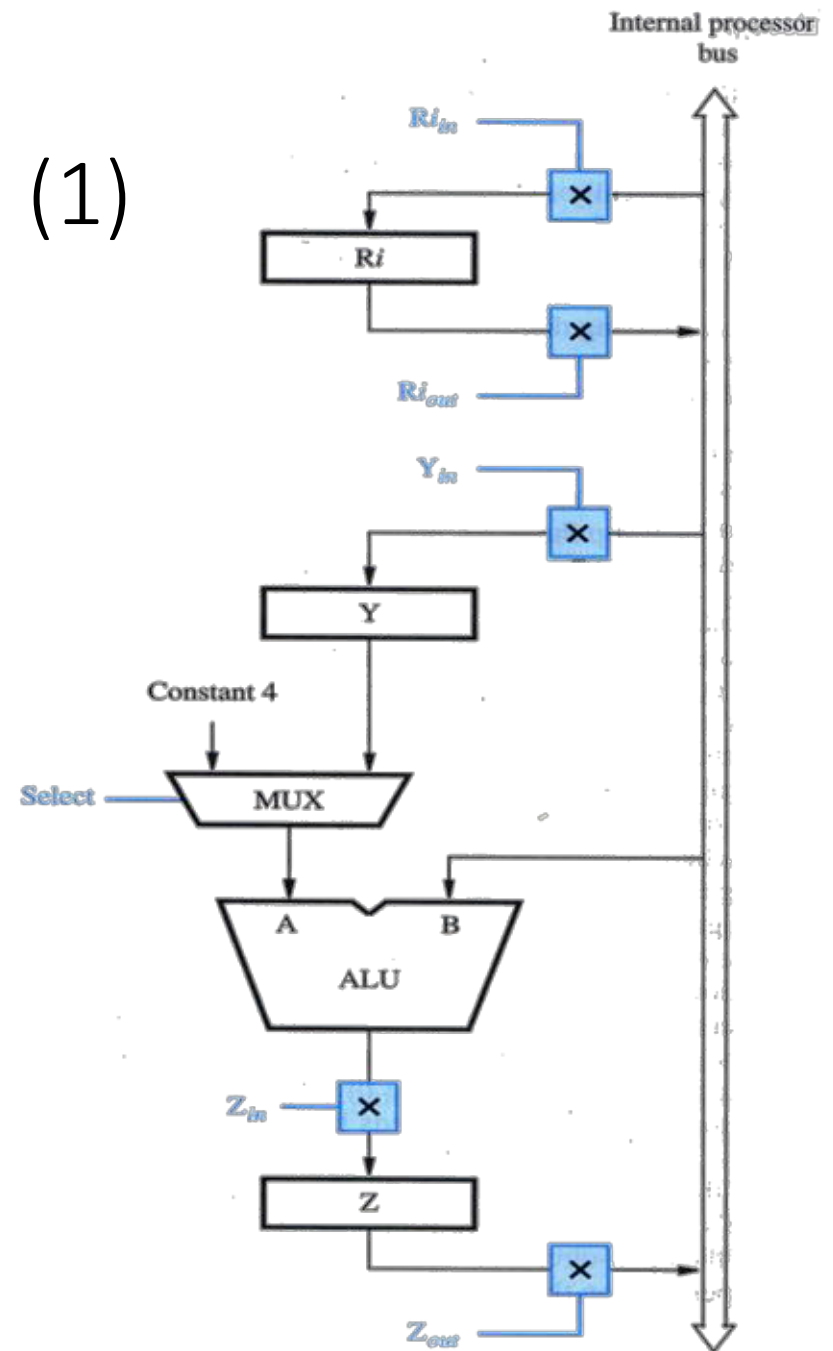


In-steps of Executing an Instruction

- **Register transfers data** between general purpose registers and ALU-registers.
- **Fetches the data** content from a memory location and loads it into a register.
- **Performs an arithmetic or a logic** operation and stores the result in a register.
- **Stores a word** from a register to a memory location.

Register Transfers Data (1)

- Each register has two control signals: R_{in} and R_{out} .
 - When R_{in} is 1, the data from a bus is loaded into a register.
 - When R_{out} is 1, the content of a register is placed on the bus.
- The following figure shows the signals that govern a data transfer operation.

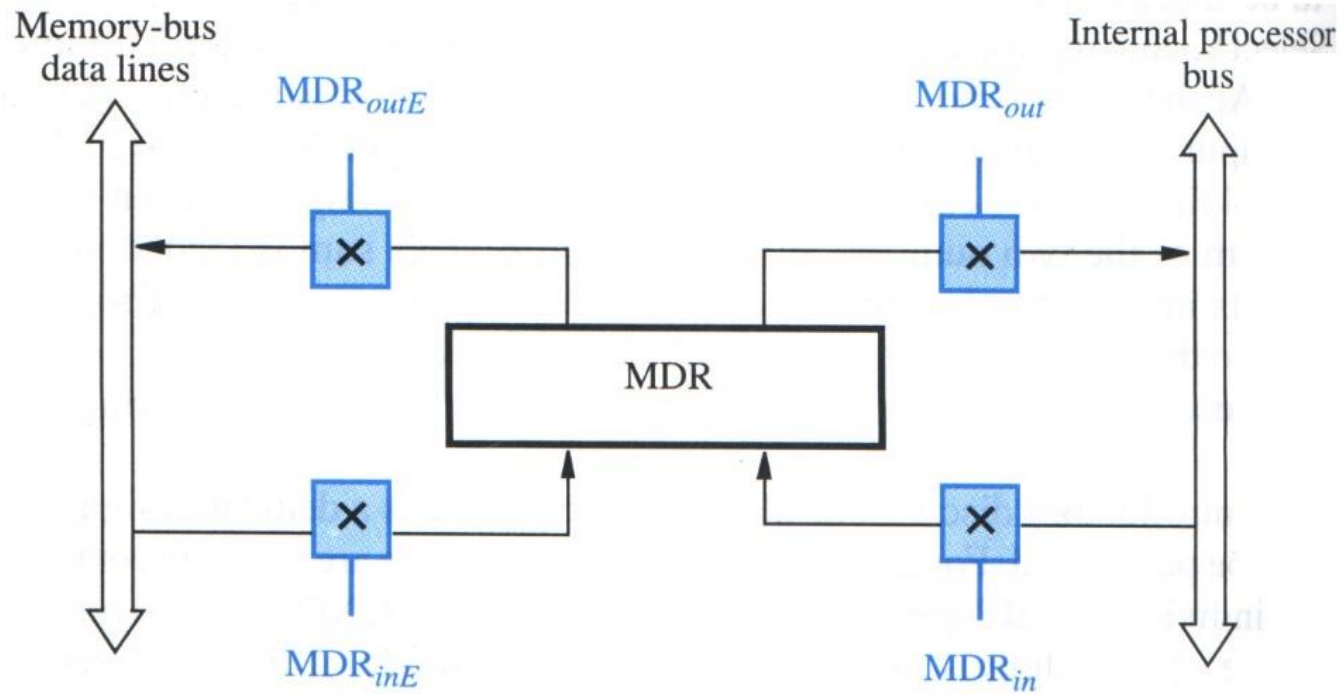
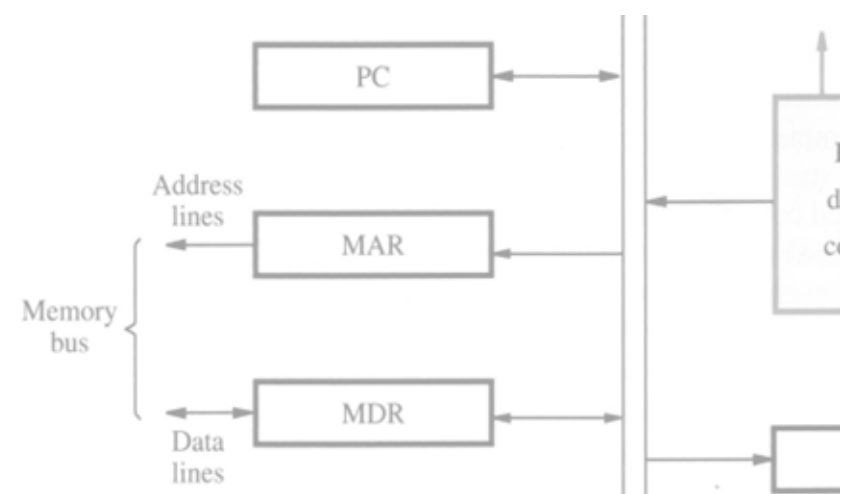


Register Transfers Data (2)

- All operations and data transfers are controlled by the processor clock.



Fetching A Word (1)



Fetching A Word (2)

- Ex Mov (R1), R2

...

MAR <- [R1]

Start a read on a memory bus

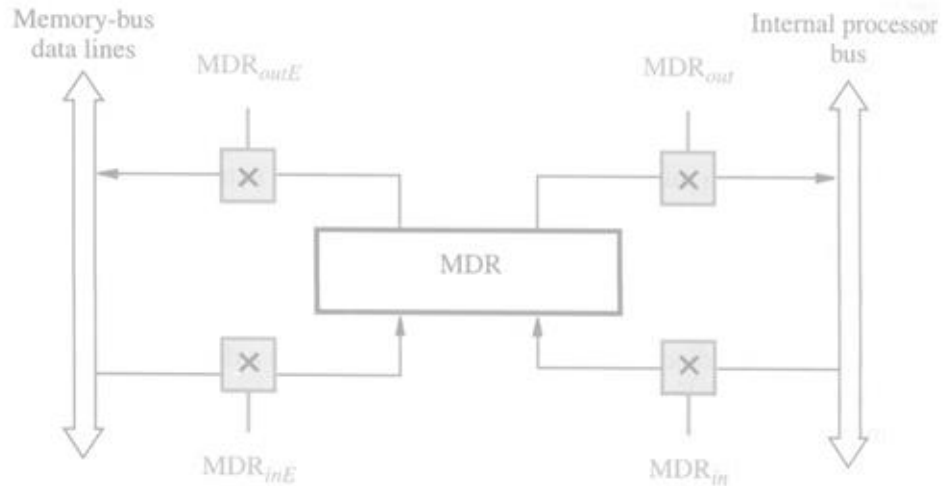
Wait for the MFC response from a memory

Load the MDR with the content on a memory bus

R2 <- MDR

...

- Note: MFC stands for Memory Function Complete.

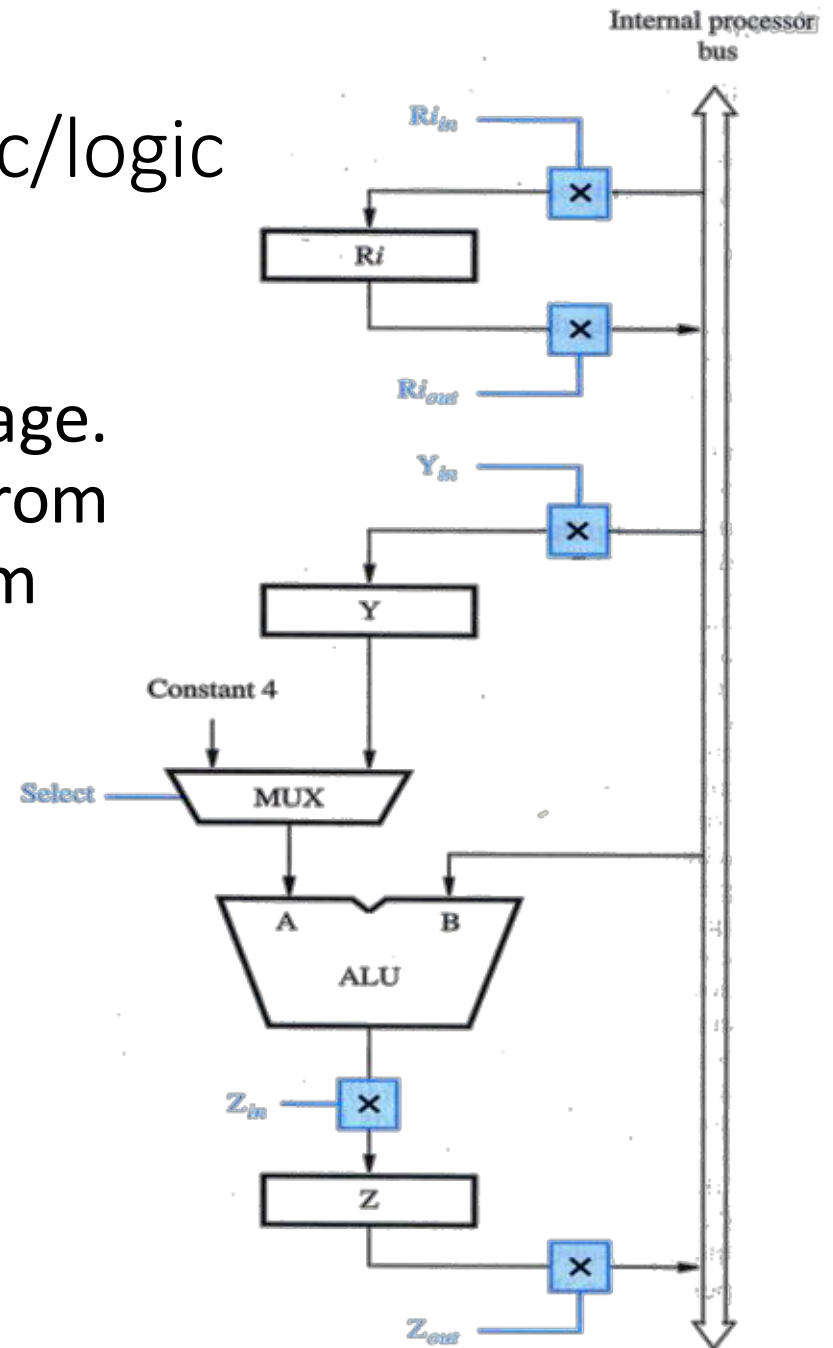


Fetching A Word (3)

- A required address are **put into** the MAR.
- A control signal will **indicate** a read operation.
- When the **data arrives**, it is stored in the MDR.
- A processor can complete one internal transfer in 1 clock. The cache hit also needs 1 clock, but when a cache miss occurs, several clocks will be needed. Thus, a processor has to wait until a control signal (**M**emory-**F**unction-**C**omplete) informs it that the read operation is completed.

Performing An Arithmetic/logic Operation (1)

- The ALU has no internal storage. ALU gets the two operands from MUX and bus. The result from the ALU is temporary stored which in register Z.
- Ex $R3 = [R1] + [R2]$ has the following sequence:
 - R1out, Yin
 - R2out, select Y, Add, Zin
 - Zout, R3in



Performing An Arithmetic/logic Operation (2)

- The content of R1 is first put in Y.
- Then MUX put the content of Y in the ALU (input A).
- At the same time, the content of R2 are put on the bus and into input B of the ALU.
- The add control line is set to one causing the addition of A and B.
- The sum is loaded into register Z.
- Finally, the content of Z is transferred to R3.

Storing A Word and Complete Instruction

- Similar to the reading operation sequence
- Ex Mov R3, #5(R1)
 - ...
 - Compute effective address $[R1] + 5$
 - MAR $\leftarrow$ EA, MDR $\leftarrow$ [R3]
 - Start a write on a memory bus
 - Wait for the MFC response from a memory
- The first operand is needed to be fetched as it resides in the memory.

Execution of a Complete Instruction

Ex. Add (R3), R1

Step	Action
1	PC _{out} , MAR _{in} , Read, Select4, Add, Z _{in}
2	Z _{out} , PC _{in} , WMFC
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMFC
6	MDR _{out} , SelectY, Add, Z _{in}
7	Z _{out} , R1 _{in} , End

Understand Stages and Data Paths

Ex. Add R4, R5, R6

Execution of Branch Instructions

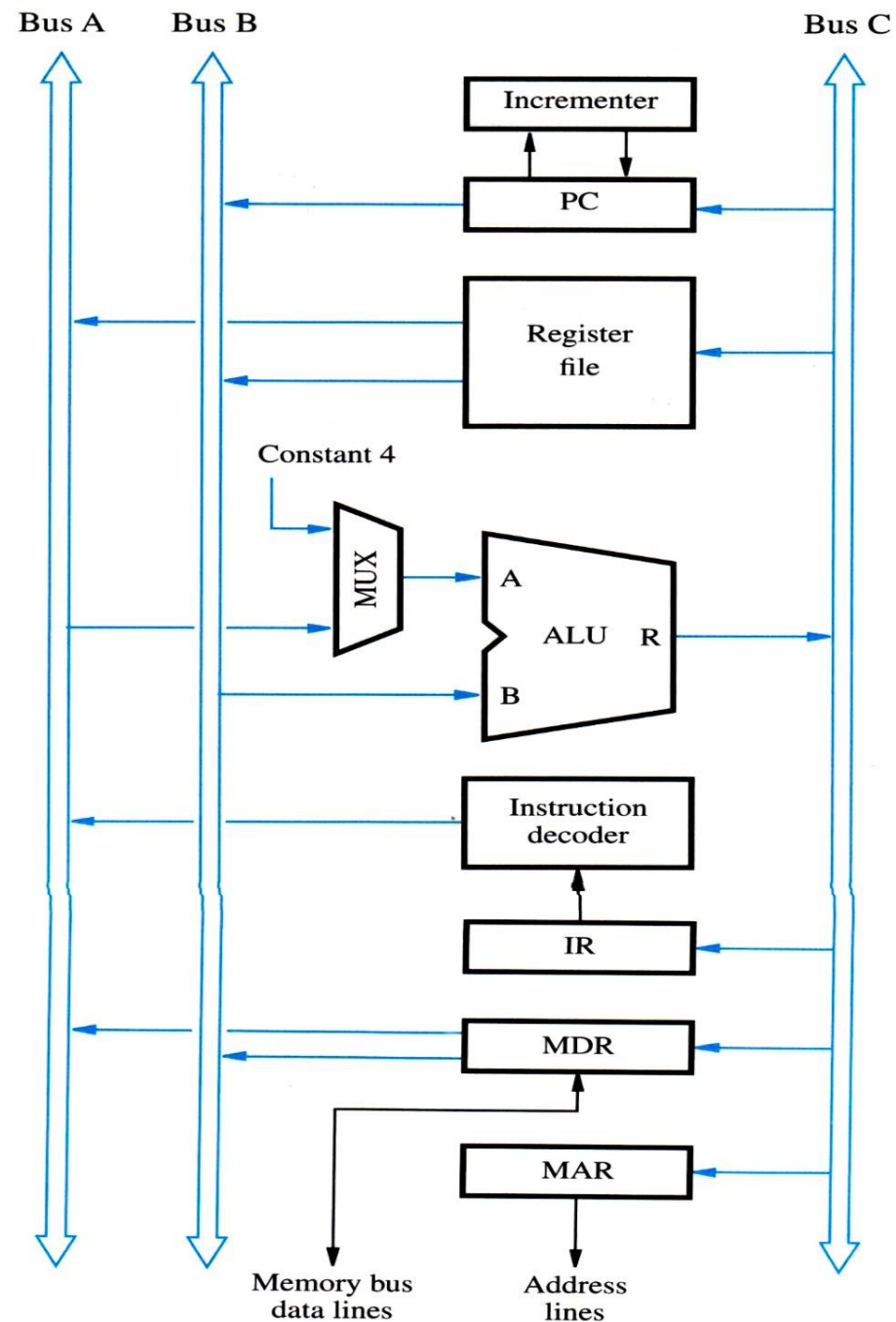
- A branch instruction replaces the contents of PC with the branch target address, which is usually obtained by adding an offset X given in the branch instruction.
- The offset X is usually the difference between the branch target address and the address immediately following the branch instruction.
- Conditional branch

Execution of Branch Instructions

Step	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMFC
3	MDR_{out} , IR_{in}
4	PC_{out} , Y_{in}
5	Offset-field-of- IR_{out} , Select Y, Add, Z_{in}
6	Z_{out} , PC_{in} , End

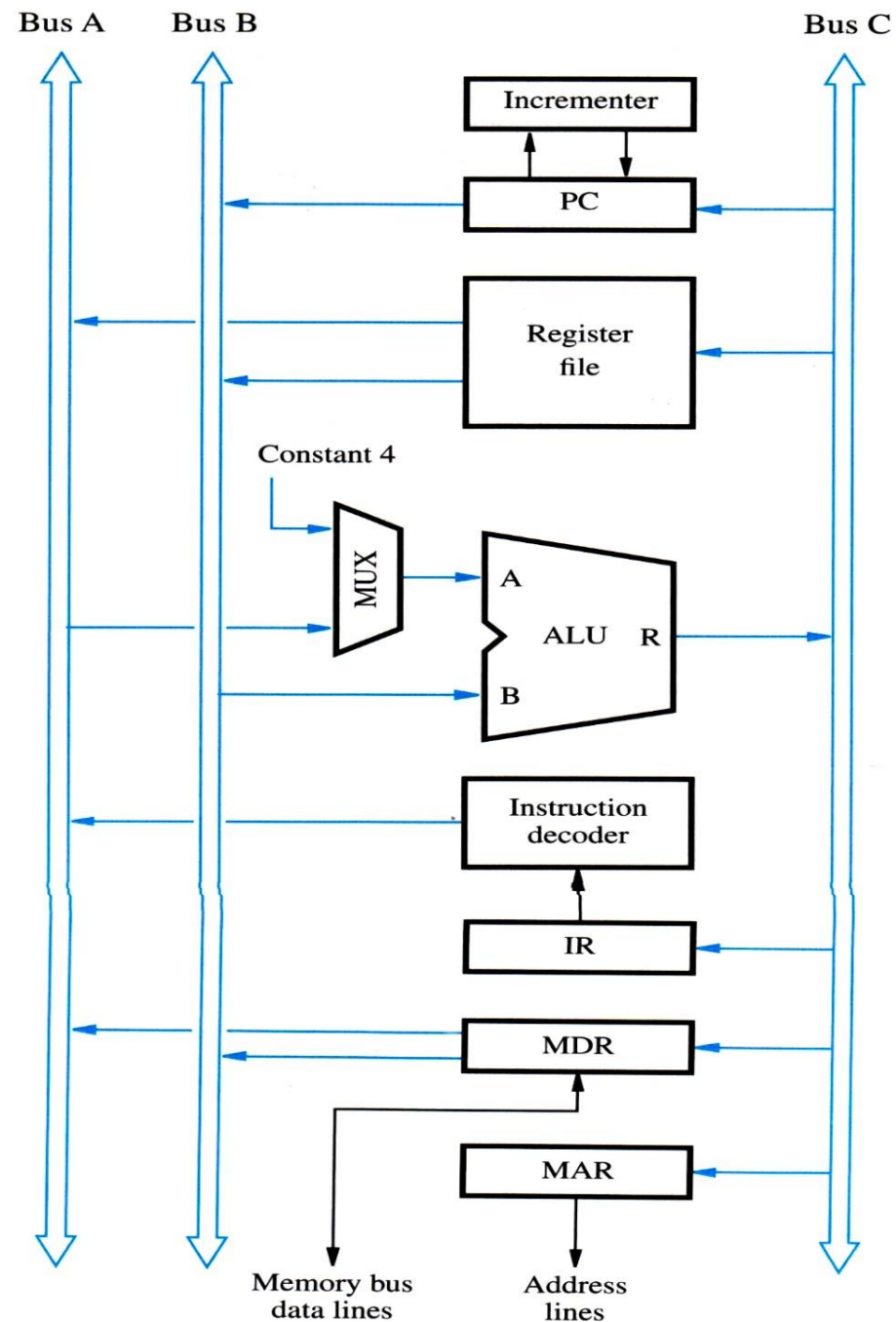
A Three-bus Organization (1)

- To reduce the number of clock used to fetch data, a processor can employ a multiple-bus organization, where several **data transfer** can take place **in parallel**.



A Three-bus Organization (2)

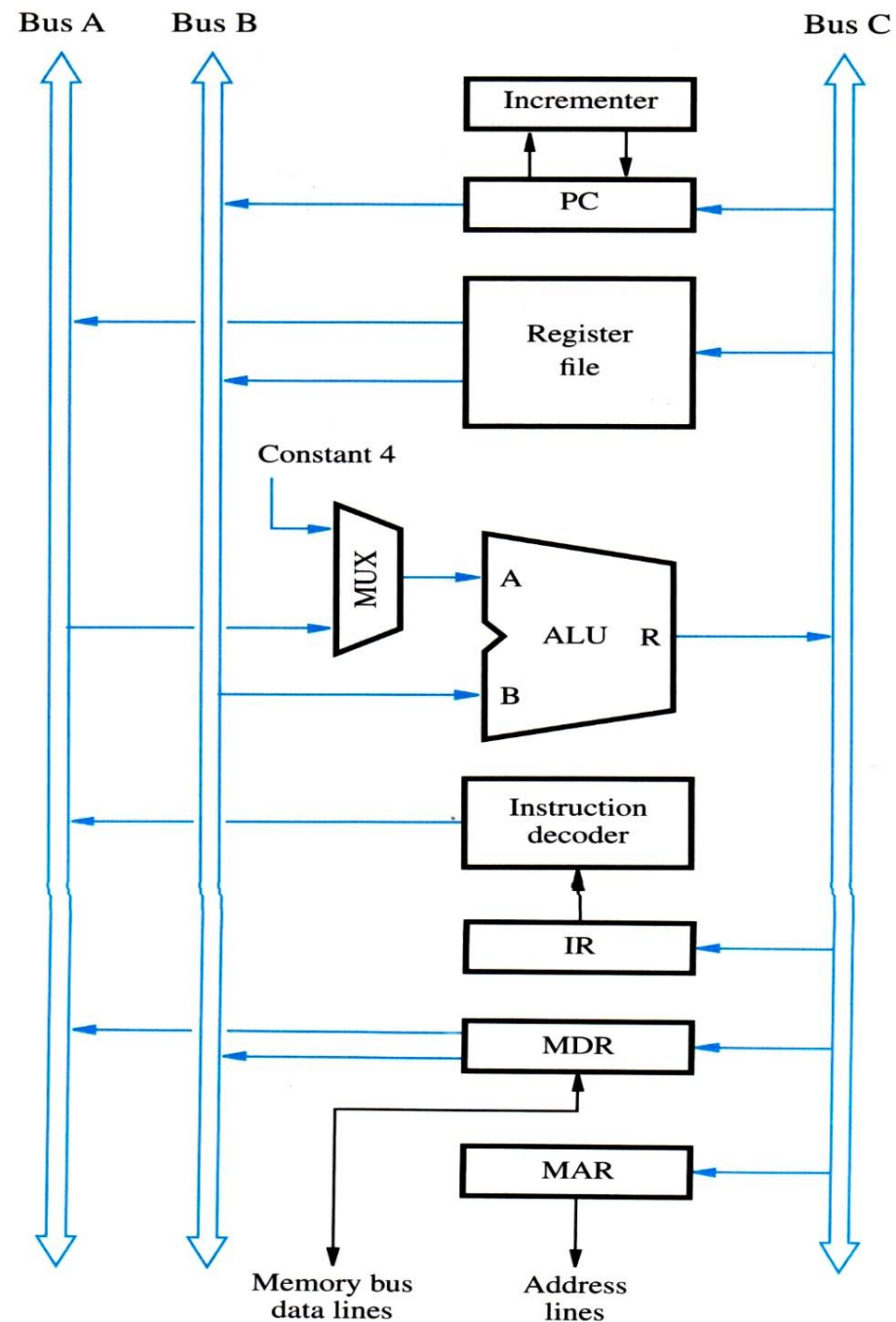
- A register is connected to all 3 buses.
- Two output buses allow two different registers to be accessed simultaneously (bus A and B).
- Bus C allows data to be loaded into another register during the same clock cycle.



A Three-bus Organization (3)

- Add R4, R5, R6

Step	Action
1	PC_{out} , $R=B$, MAR_{in} , Read, IncPC
2	WMFC
3	MDR_{outB} , $R=B$, IR_{in}
4	$R4_{outA}$, $R5_{outB}$, SelectA, Add, $R6_{in}$, End



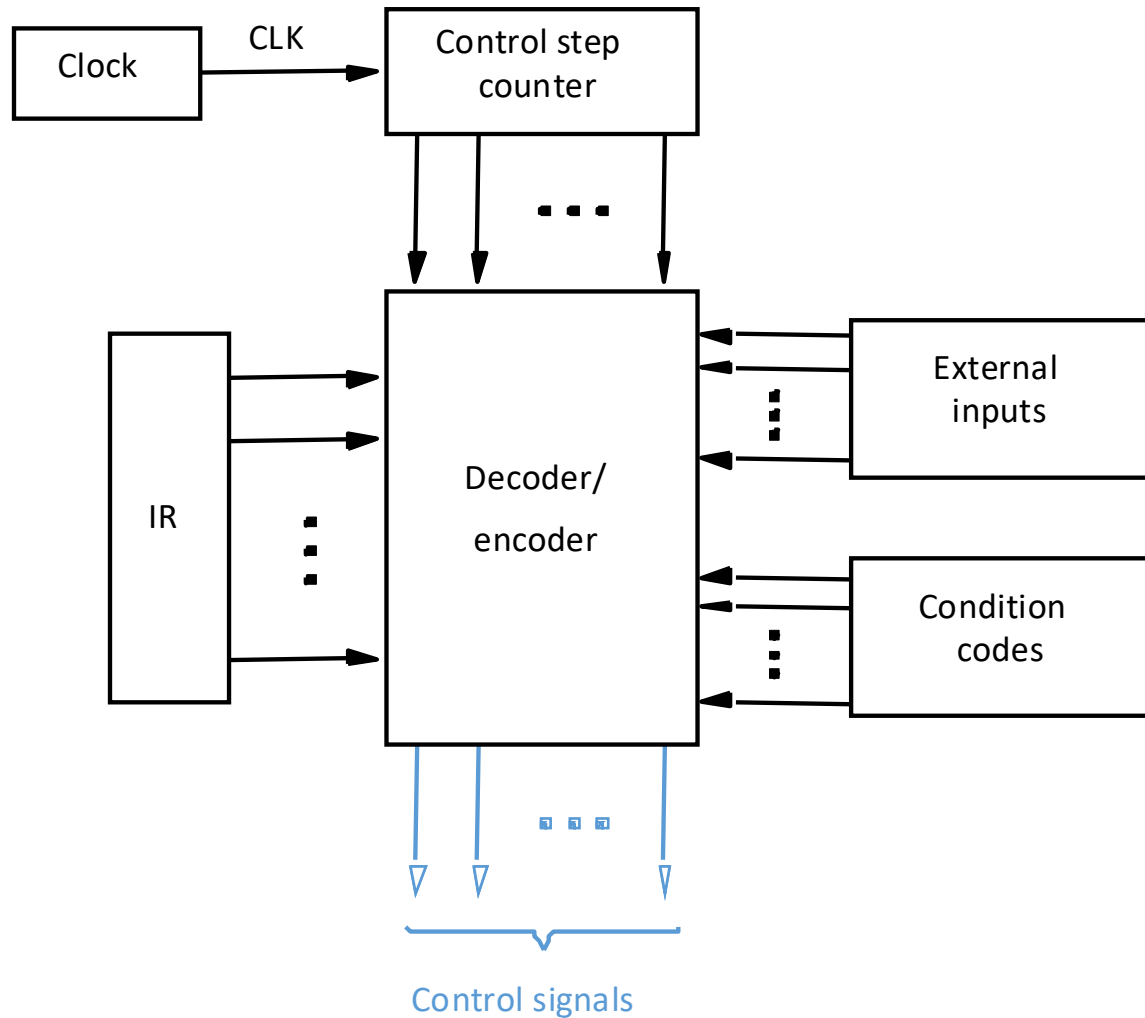
A Three-bus Organization (4)

- The ALU uses bus A and B for the operands and bus C to transfer the result.
- An **incrementer** unit is used to increment the content of the PC (by 4). The incrementer eliminates the need for the ALU in incrementing the PC's content. However, the constant 4 at the ALU input is still useful as it can be used to increment other addresses, such as the memory address in *loadmultiple* and *storemultiple* instructions.
- The ALU can **pass** one of its input unmodified to bus C. The operation can be represented as $R=A$ or $R=B$.
- The bus structure **avoids** the need for temporary register (Y, Z, TEMP).

Decode/Encode and Control Logic

- To execute instructions, the processor must have some means of generating the control signals needed in the proper sequence.
- Two categories: **hardwired control** and **microprogrammed control**
- Hardwired system can operate at high speed; but with little flexibility.

Hardwired control (1)

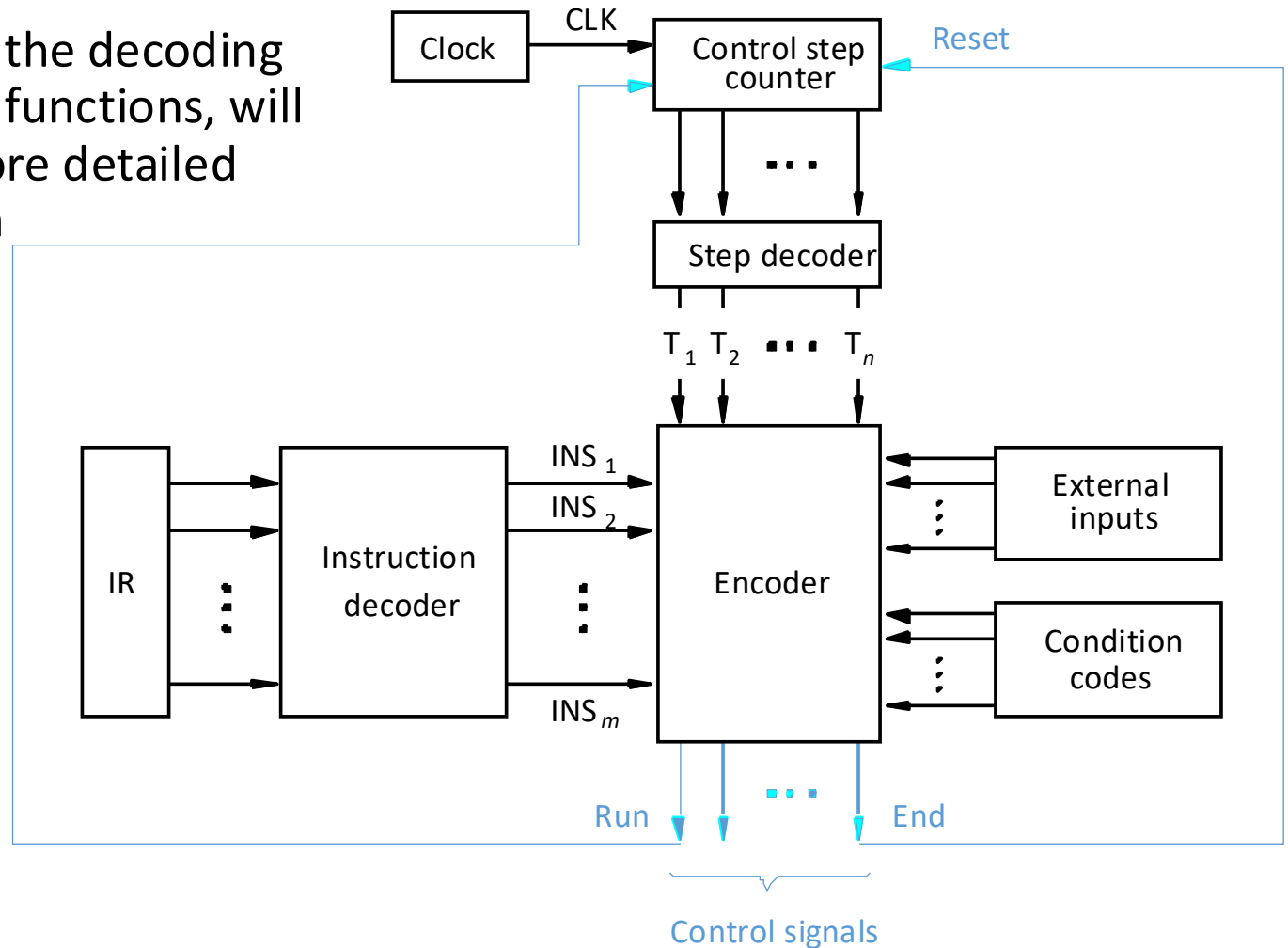


Hardwired Control (2)

- The sequence of control signals step is complete in one clock period.
- A counter may be used to keep track of the control steps, as shown in previous slide. Each state, or count, of this counter corresponds to one control step.
- The required control signals are determined by the following information:
 1. Contents of the control step counter
 2. Contents of the instruction register
 3. Contents of the condition code flags
 4. External input signals, such as MFC and interrupt requests

Detailed Block Description (1)

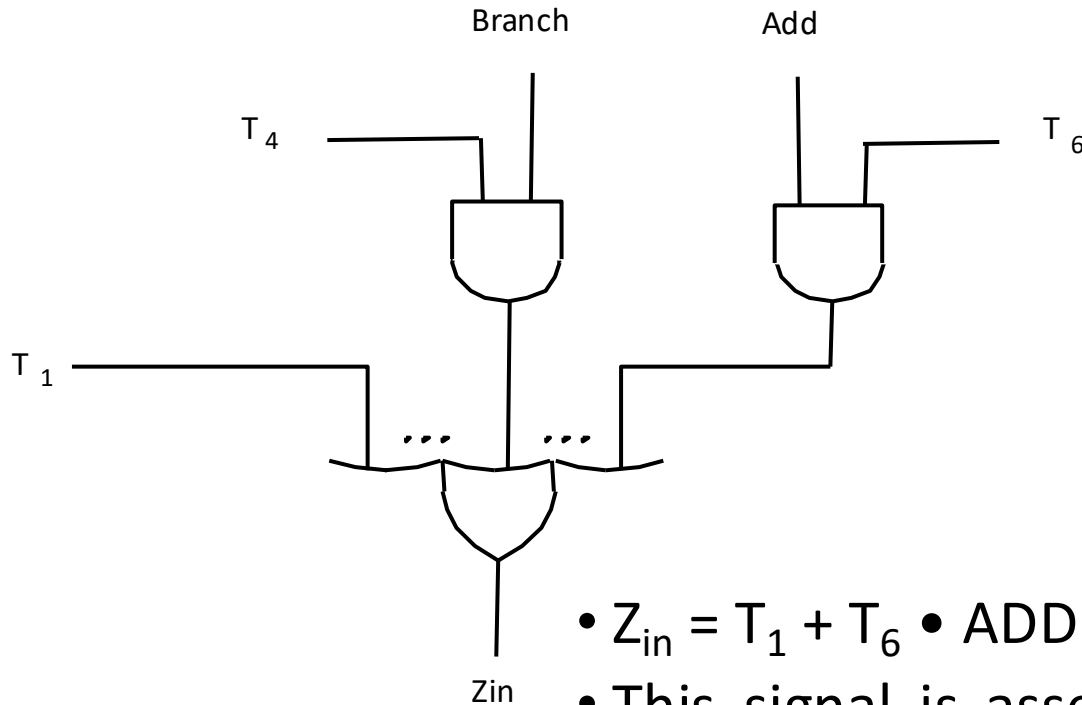
- By separating the decoding and encoding functions, will obtain the more detailed block diagram



Detailed Block Description (2)

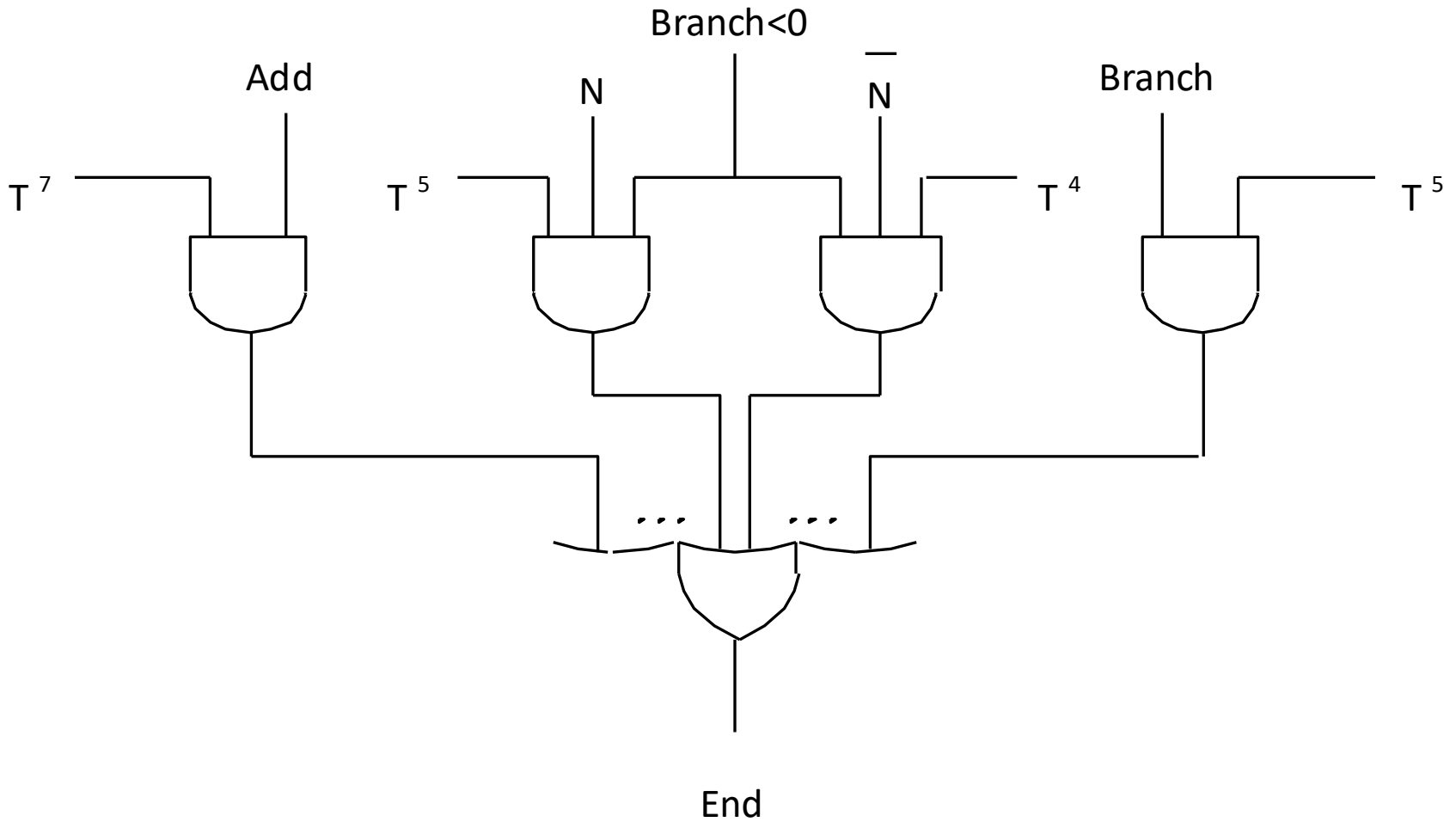
- The step decoder provides a separate signal line for each step, or time slot, in the control sequence.
- Similarly, the output of the instruction decoder consists of a separate line for each machine instruction. For any instruction loaded in the IR, one of the output lines INS_1 through INS_m is set to 1, and all other lines are set to 0.
- The input signals to the encoder block in diagram combined to generate the individual control signals Y_{in} , PC_{out} , Add , End , and so on.

Generating Z_{in}



- $Z_{in} = T_1 + T_6 \bullet \text{ADD} + T_4 \bullet \text{BR} + \dots$
- This signal is asserted during time slot T1 for all instructions, during T6 for an Add instruction, during T4 for an unconditional branch instruction, and so on. The logic function for Z_{in} is derived from the control sequences

Generating of The End Control Signal (1)



Generating of The End Control Signal (2)

- $\text{End} = T_7 \bullet \text{ADD} + T_5 \bullet \text{BR} + (T_5 \bullet \text{N} + T_4 \bullet \text{N}) \bullet \text{BRN} + \dots$
- The End signal starts a new instruction fetch cycle by resetting the control step counter to its starting value.
- Set to 1, RUN causes the counter to be incremented by one at the end of every clock cycle. When RUN is equal to 0, the counter stops counting.
- This is needed whenever the WMFC signal is issued, to cause the processor to wait for the reply from the memory.

Microprogram Control

- The control signals are generated by a program similar to a machine language program.
- Each binary string is a **micro instruction** or **control word** (CW).
- A sequence of CWs corresponding to the control sequence of machine instructions constitutes the micro-routine.

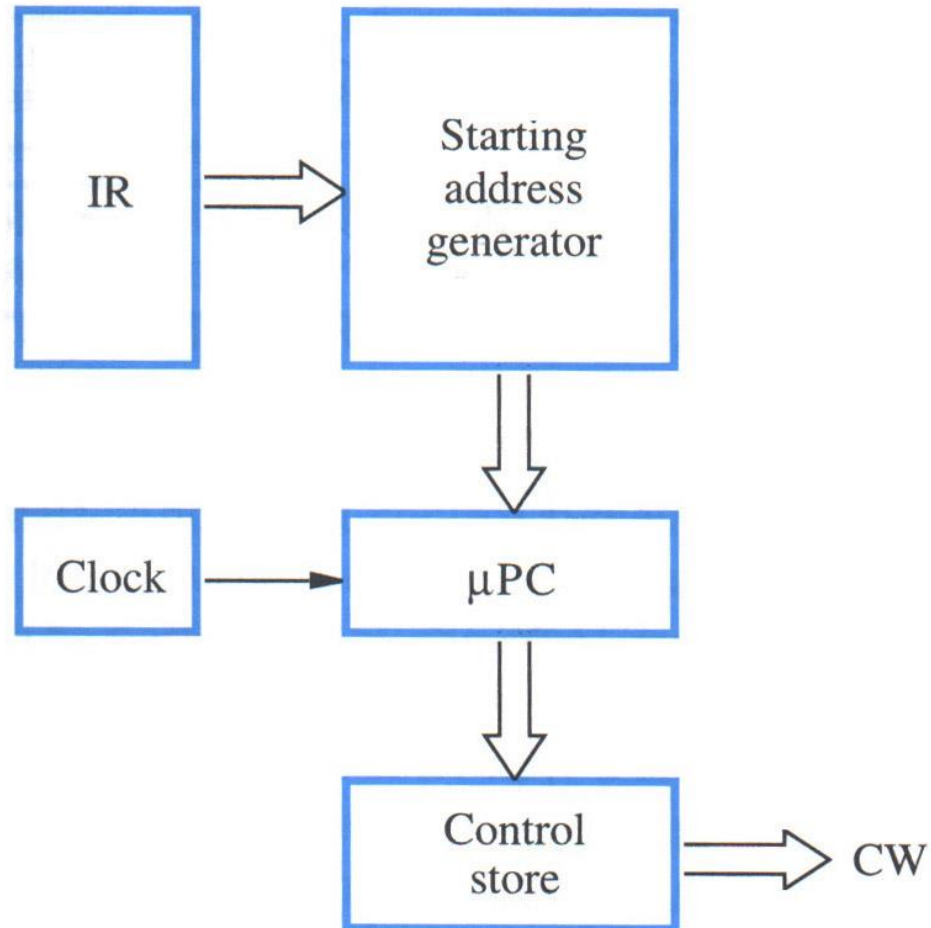
Micro Instructions (1)

Micro - instruction	·	PC _{in}	PC _{out}	MAR _{in}	Read	MDR _{out}	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	R1 _{out}	R1 _{in}	R3 _{out}	WMFC	End	:
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	

Micro Instructions (2)

- Control words (micro instructions) are stored in the **control store**.
- The instruction decoder and control unit can generate the control signal by sequentially reading control words of the corresponding micro routine from the control store.

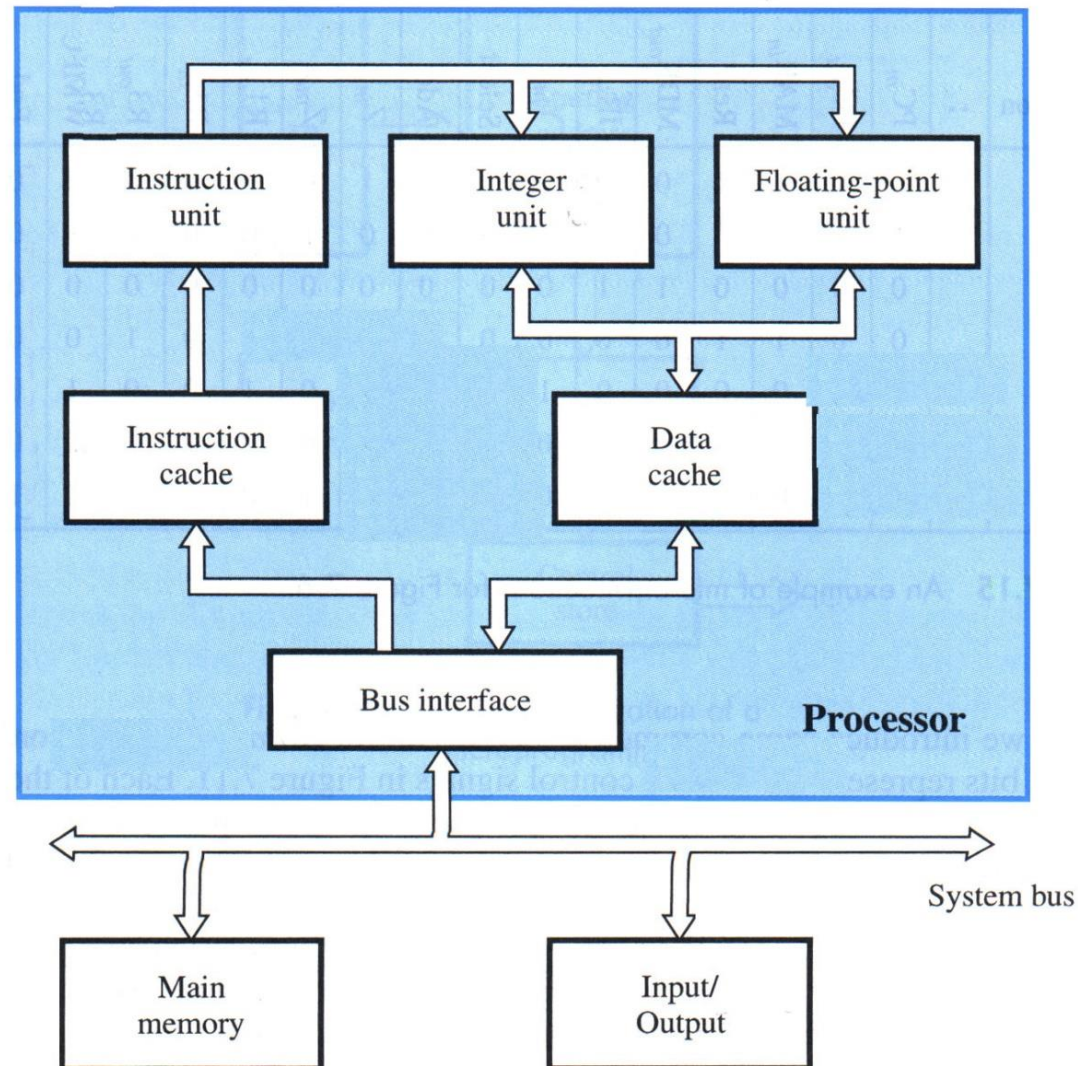
The Control Unit Organization (1)



The Control Unit Organization (2)

- The **Micro-PC** is used to read control words sequentially from the control store.
- A set of control words are processor dependent.
- The assembler translates the assembly code into machine code, which is binary. Each word in a machine code tell the processor what to do.
 - For example, 0100 0000 = raise the PCin signal.
- Thus, the execution of a program is directed by a set of control words generated by the assembler.

A Complete Processor (1)



A Complete Processor (2)

- An instruction unit is used to fetch instructions from an instruction cache or a memory.
- There are two separate processing units: integer and floating point. Both units use the data cache.
- The bus interface unit connects the processor to the system bus.

Questions

Supplementary section

Partial Format for the Microinstructions (1)

Microinstruction

F1	F2	F3	F4	F5
F1 (4 bits)	F2 (3 bits)	F3 (3 bits)	F4 (4 bits)	F5 (2 bits)
0000: No transfer	000: No transfer	000: No transfer	0000: Add	00: No action
0001: PC <i>out</i>	001: PC <i>in</i>	001: MAR <i>in</i>	0001: Sub	01: Read
0010: MDR <i>out</i>	010: IR <i>in</i>	010: MDR <i>in</i>	⋮	10: Write
0011: Z <i>out</i>	011: Z <i>in</i>	011: TEMP <i>in</i>	1111: XOR	
0100: R0 <i>out</i>	100: R0 <i>in</i>	100: Y <i>in</i>	⏟	
0101: R1 <i>out</i>	101: R1 <i>in</i>		16 ALU functions	
0110: R2 <i>out</i>	110: R2 <i>in</i>			
0111: R3 <i>out</i>	111: R3 <i>in</i>			
1010: TEMP <i>out</i>				
1011: Offset <i>out</i>				
F6	F7	F8	...	
F6 (1 bit)	F7 (1 bit)	F8 (1 bit)		
0: SelectY	0: No action	0: Continue		
1: Select4	1: WMFC	1: End		

Partial Format for the Microinstructions (2)

- A straightforward way to structure micro instructions is to assign one bit position to each control signal. However, this is very inefficient.
- The length of the microinstructions can be reduced easily.
- Most signals are not needed simultaneously, and many signals are mutually exclusive. For e.g., only one function of the ALU can be activated at a time.
 - The source for a data transfer must be unique because it is not possible to gate the contents of two different registers onto the bus at the same time.
 - Read and Write signals to the memory cannot be active simultaneously.

Partial Format for the Microinstructions (3)

- This suggests that signals can be grouped so that all mutually exclusive signals are placed in the same group. Thus, at most one *micro operation* per group is specified in any microinstruction.
- Then it is possible to use a binary coding scheme to represent the signals within a group.
- For example, four bits suffice to represent the 16 available functions in the ALU. Register output control signals can be placed in a group consisting of PCout, MDROUT, Zout, Offsetout, R0out, R1out, R2out, R3out, and TEMPout. Any one of these can be selected by a unique 4-bit code. a

Further Improvement (1)

- Enumerate the patterns of required signals in all possible microinstructions. Each meaningful combination of active control signals can then be assigned a distinct code.
- Highly encoded schemes that use compact codes to specify only a small number of control functions in each microinstruction are referred to as a ***vertical organization***.
- On the other hand, the minimally encoded scheme, in which many resources can be controlled with a single microinstruction, is called a ***horizontal organization***.

Further Improvement (2)

- The horizontal approach is useful when a higher operating speed is desired and when the machine structure allows parallel use of resources.
- The vertical approach results in considerably slower operating speeds because more microinstructions are needed to perform the desired control functions. Although fewer bits are required for each microinstruction, this does not imply that the total number of bits in the control store is smaller. The significant factor is that less hardware is needed to handle the execution of microinstructions.
- Horizontal and vertical organizations represent the two organizational extremes in micro programmed control.